Peblo Flutter / Swift Developer Intern Challenge: "The AI Story Buddy & Quiz Component"

Company Introduction – Peblo

Peblo is a mission-driven, seed-funded edutainment startup on a mission to reshape early and middle-grade education. We blend immersive storytelling, interactive game design, pedagogy, and cutting-edge AI to create personalized learning journeys for children in India and beyond. At Peblo, education meets entertainment to foster joyful, emotionally resonant experiences that adapt to every child's unique curiosity and pace.
We care deeply about craft, and we'd love for you to get a feel for what we're building before we go further.
Visit our YouTube channel that walks through what we're planning to offer in the app and how we approach it creatively:
https://www.youtube.com/@peblotv
Also look at our other other Socials

- Website: www.mypeblo.com
- LinkedIn: Linkedin.com/company/mypeblo
- Instagram: Instagram.com/mypeblo

Assessment Overview

At Peblo, we blend immersive storytelling, interactive gamification, and AI to create joyful learning journeys for children. For this challenge, you'll build a mini-feature from the Peblo ecosystem: an **AI Story Buddy** that reads a short story snippet aloud to a child and follows up with an interactive quiz question.

You may build this in **Flutter** or **Swift (Native iOS)** or BOTH.

Our primary audience is children in India, many on **mid-range Android devices (≈3GB RAM)**. Build with that constraint in mind: smooth, lightweight, and resilient on modest hardware.

The Task

Build a **single-screen mobile app** with a gamified, kid-friendly UI that includes a text-to-speech narration trigger and a data-driven interactive quiz.

We've attached a **wireframe** showing the approximate layout and our brand colours. Treat it as a guide for structure, not a pixel-perfect spec — the visual polish and the "joyful, child-friendly"

feel are yours to bring. We want to see both that you can build to a layout *and* that you have a sense for what delights a young child.

UI/UX Wireframe

1. The Core UI/UX (kid-friendly)

- A vibrant, visually engaging screen following the attached wireframe layout.
- An **AI Buddy character** (any cute placeholder illustration/asset is fine).
- A prominent **"Read Me a Story"** button.
- A **story text card** that displays the narrative text.

2. Audio & TTS Integration

- When the user taps **"Read Me a Story"**, trigger text-to-speech narration of the story text.
  - Acceptable: the device's **native TTS engine** (`AVSpeechSynthesizer` on iOS, `flutter_tts` on Flutter).
  - **Bonus:** integrate a real free-tier API such as ElevenLabs.
- **Handle the real-world states properly:**
  - Show a **loading/preparing state** while audio is being fetched or prepared.
  - Handle the **failure case** gracefully — if TTS fails or there's no network, show a friendly message and let the user retry. Don't let the app hang or crash.

The story text to narrate:

> *"Once upon a time, a clever little robot named Pip lost his shiny blue gear in the Whispering Woods..."*

3. The Interactive Quiz Engine (data-driven)

This is the most important part. **Do not hardcode the question.** Render the quiz from the JSON object below, as if it were served by our backend. Assume the next question we send might have **a different number of options (3, 4, or 5)** or different text — your renderer should handle that without code changes.

```
json

{

  "question": "What colour was Pip the Robot's lost gear?",

  "options": ["Red", "Green", "Blue", "Yellow"],

  "answer": "Blue"

}
```

- As soon as the audio finishes, smoothly **reveal the quiz** built from this JSON.
- **Interaction:**
    - On a **wrong** answer: shake the card (with haptic and/or visual feedback) and let the child try again.
    - On the **correct** answer: trigger a celebratory moment (confetti, the Buddy smiling, a happy state change) and show a **"Success"** state.

Technical Requirements & Focus Areas

If you choose Flutter:

- **State management:** use **Provider, Riverpod, or BLoC** to manage audio playback states, quiz state, and UI animations.
- **Performance:** animations (shake, confetti) must run smoothly at ~60fps. Avoid unnecessary widget rebuilds.
- **Data-driven rendering:** the quiz UI must be generated from the JSON, not hardcoded.

If you choose Swift (iOS Native):

- **UI framework:** SwiftUI preferred, using `@State`/`@Binding` for reactive updates.
- **TTS:** use `AVFoundation` (`AVSpeechSynthesizer`) for narration.
- **Memory management:** no retain cycles or leaks when handling audio-completion closures/delegates.
- **Data-driven rendering:** the quiz UI must be generated from the JSON, not hardcoded.

Submission Guidelines

Please submit within **3 days**:

1. Open the Google Form: https://forms.gle/6re5JGsiUgyGBc5r7
2. Fill in your **personal details**.
3. **GitHub repository link** — clean, documented code with a clear project structure.
4. **A short README** covering:
    - Which framework you chose and why.
    - How you managed the **transition state** between audio ending and quiz appearing.
    - How you built the quiz to be **data-driven** (how it handles a different question / option count).
    - Your **caching approach** (and how you'd cache remote audio if applicable).
    - How you **handled audio loading and failure states**.
    - Your **performance profiling**: what you measured, what you changed, before/after (with the frame-timing screenshot).
    - How you **optimized to stay lightweight** on mid-range Android devices.

    - **AI usage & judgment:** Where did you use AI assistance? Name one suggestion it gave that you **rejected or changed**, and why. What did you try that **didn't work**, and how did you resolve it?
5. **A screen recording —** a short video showing the full flow end-to-end (audio playing → quiz appearing → wrong-answer feedback → success state).
6. **What We're Evaluating**

- Clean, readable, well-structured code.
- Smooth, joyful, kid-appropriate UI and animation.
- Correct, leak-free handling of audio playback and state transitions.
- A genuinely **data-driven** quiz renderer (not hardcoded).
- Production-mindedness: loading states, error handling, performance on modest devices.

We're excited to see what you build. Have fun with it - this is the kind of delightful, child-first work you'd be doing at Peblo every day.